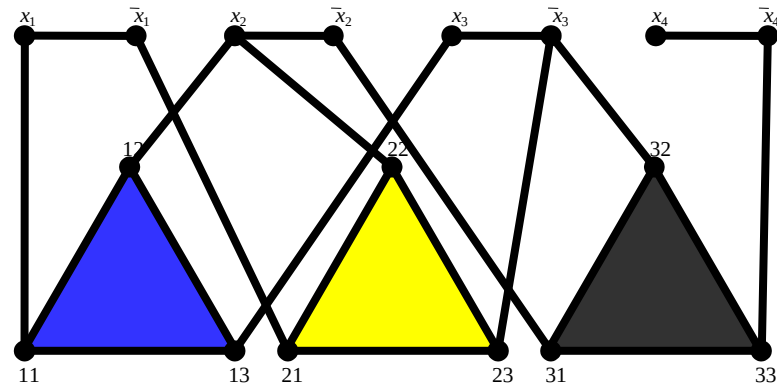


Lecture 12: P & NP



Learning Outcome

◆ P and NP

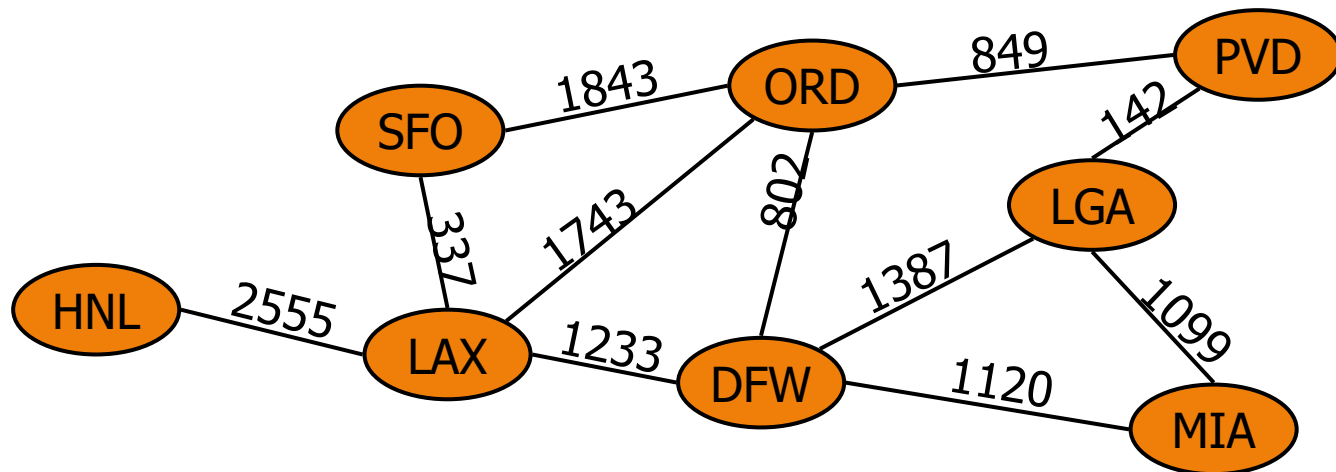
- Definition of P
- Definition of NP
- Alternate definition of NP

◆ NP-completeness

- Definition of NP-hard and NP-complete
- The Cook-Levin Theorem

Running Time Revisited

- ◆ Input size, n
 - To be exact, let n denote the number of **bits** in a nonunary (binary) encoding of the input
- ◆ A time bound of the form $O(n^k)$, for some fixed k , is called polynomial time.
- ◆ All the polynomial-time algorithms studied so far in this course run in polynomial time using this definition of input size.
 - Exception: any pseudo-polynomial time algorithm



Dealing with Hard Problems

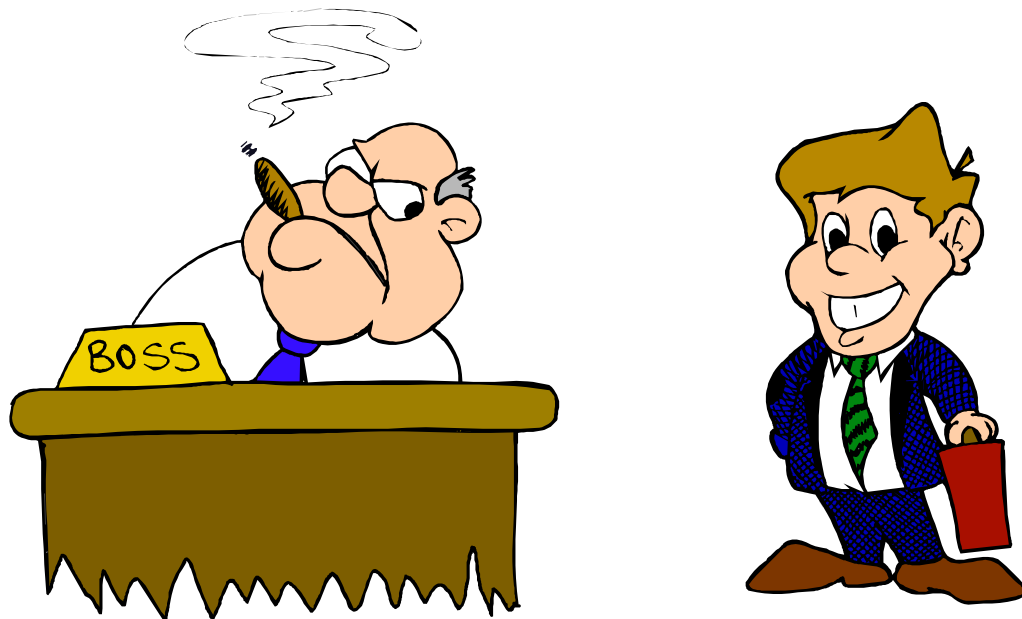
◆ What to do when we find a problem that looks hard...



I couldn't find a polynomial-time algorithm;
I guess I'm too dumb.

Dealing with Hard Problems

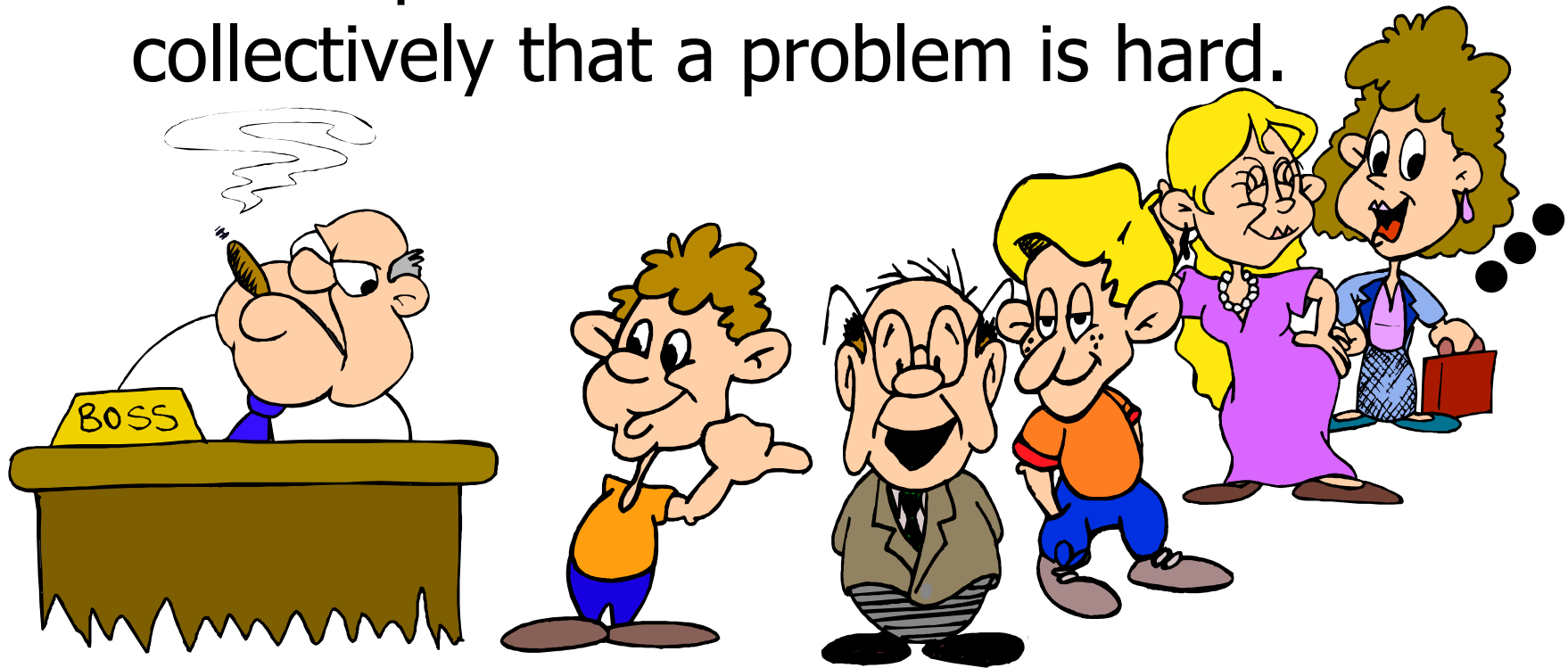
◆ Sometimes we can prove a strong lower bound... (but not usually)



I couldn't find a polynomial-time algorithm,
because no such algorithm exists!

Dealing with Hard Problems

◆ NP-completeness let's us show collectively that a problem is hard.



I couldn't find a polynomial-time algorithm,
but neither could all these other smart people.

Polynomial-Time Decision Problems



- ◆ To simplify the notion of “hardness,” we will focus on the following:
 - Polynomial-time n^k as the cut-off for efficiency
 - Decision problems: output is 1 or 0 (“yes” or “no”)
 - ◆ Examples:
 - ◆ Does a given graph G have an Euler tour?
 - ◆ Does a text T contain a pattern P ?
 - ◆ Does an instance of 0/1 Knapsack have a solution with benefit at least K ?
 - ◆ Does a graph G have an MST with weight at most K ?

Problems and Languages



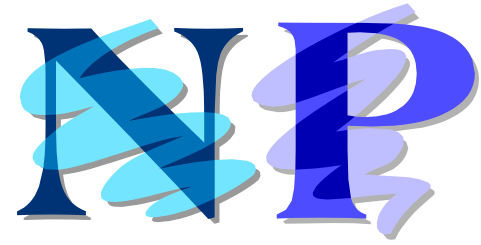
- ◆ A **language** L is a set of strings defined over some alphabet Σ
- ◆ Every decision algorithm A defines a language L
 - L is the set consisting of every string x such that A outputs “yes” on input x .
 - We say “ A **accepts** x ” in this case
 - ◆ Example:
 - ◆ If A determines whether or not a given graph G has an Euler tour, then the language L for A is all graphs with Euler tours.

The Complexity Class P

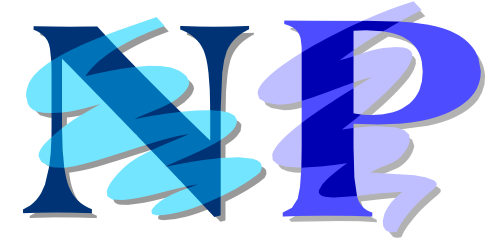


- ◆ A **complexity class** is a collection of languages (problems)
- ◆ P is the complexity class consisting of all languages that are accepted by **polynomial-time deterministic** algorithms
- ◆ P contains all decision problems that can be solved by a deterministic Turing machine in polynomial time.

The Complexity Class NP



- ◆ We say that an algorithm is non-deterministic if it uses the following operation:
 - **Choose(*b*)**: chooses a bit *b*
 - Can be used to choose an entire string *y* (with $|y|$ choices)
- ◆ We say that a non-deterministic algorithm **A** **accepts** a string *x* if there exists some sequence of **choose** operations that causes **A** to output “yes” on input *x*.
- ◆ NP is the complexity class consisting of all languages accepted by **polynomial-time non-deterministic** algorithms.

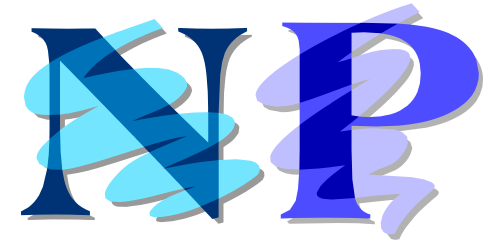


NP example

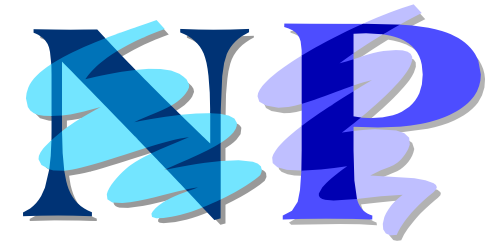
- ◆ Problem: Decide if a graph has an MST of weight K
- ◆ Algorithm (Not Prim's or Kruskal's):
 1. Non-deterministically choose a set T of $n-1$ edges
 2. Test that T forms a spanning tree
 3. Test that T has weight at most K
- ◆ Analysis: Testing takes $O(n+m)$ time, so this algorithm runs in polynomial time.

The Complexity Class NP

Alternate Definition



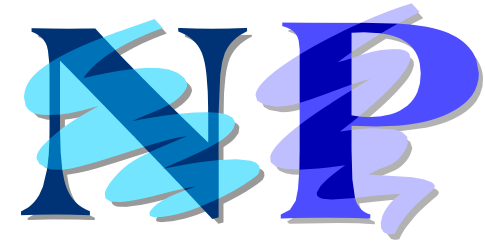
- ◆ We say that an algorithm B **verifies** the acceptance of a language L if and only if, for any x in L, there exists a certificate y such that B outputs “yes” on input (x,y) .
- ◆ NP is the complexity class consisting of all languages verified by **polynomial-time** algorithms.
- ◆ We know: P is a subset of NP.
- ◆ Major open question: $P=NP$? \$1 million prize.
- ◆ Most researchers believe that P and NP are different.



NP example

- ◆ Problem: Decide if a graph has an MST of weight K
- ◆ Verification Algorithm:
 1. Use as a certificate, y , a set T of $n-1$ edges
 2. Test that T forms a spanning tree
 3. Test that T has weight at most K
- ◆ Analysis: Verification takes $O(n+m)$ time, so this verification algorithm runs in polynomial time.

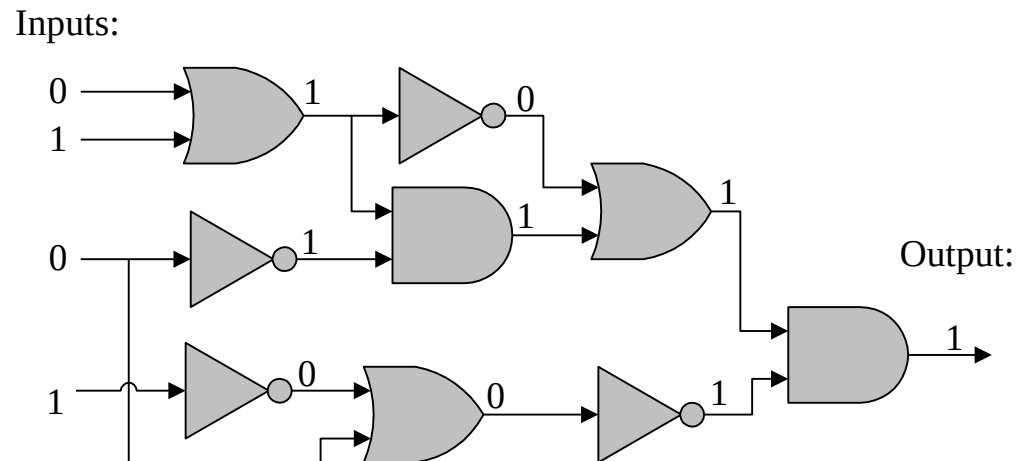
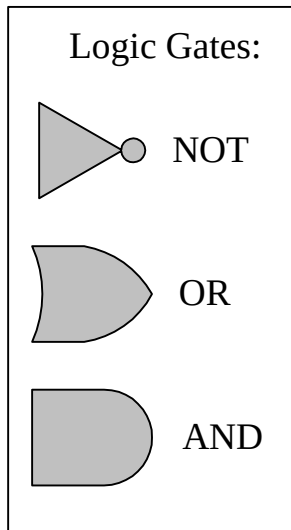
Equivalence of the Two Definitions



- ◆ Suppose A is a non-deterministic algorithm
 - ⑩ Let y be a certificate consisting of all the outcomes of the choose steps that A uses
 - ⑩ We can create a verification algorithm that uses y instead of A 's choose steps
 - ⑩ If A accepts on x , then there is a certificate y that allows us to verify this (namely, the choose steps A made)
 - ⑩ If A runs in polynomial-time, so does this verification algorithm
- ◆ Suppose B is a verification algorithm
 - ◆ Non-deterministically choose a certificate y
 - ◆ Run B on y
 - ◆ If B runs in polynomial-time, so does this non-deterministic algorithm

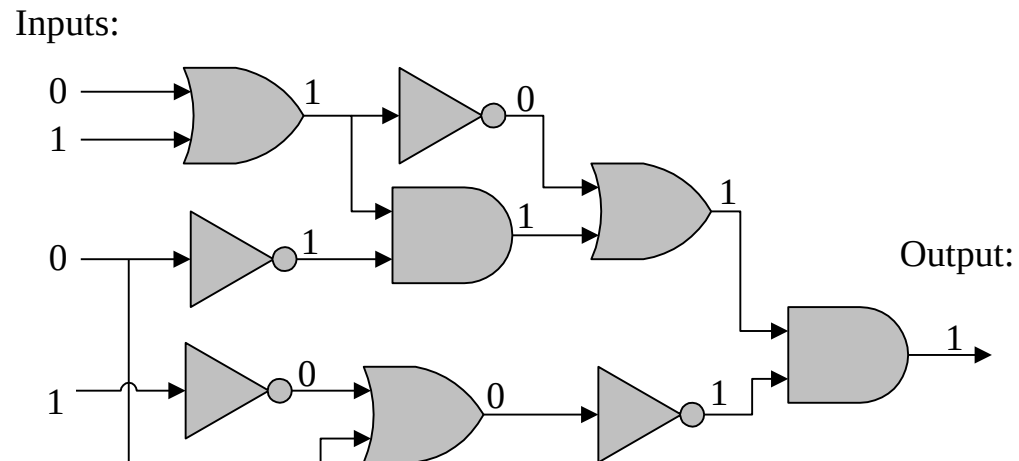
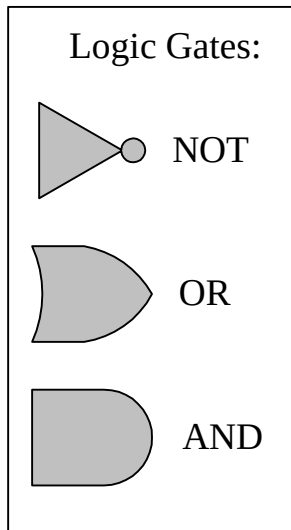
CIRCUIT-SAT

- ◆ A Boolean circuit is a circuit of AND, OR, and NOT gates; the CIRCUIT-SAT problem is to determine if there is an assignment of 0's and 1's to a circuit's inputs so that the circuit outputs 1.



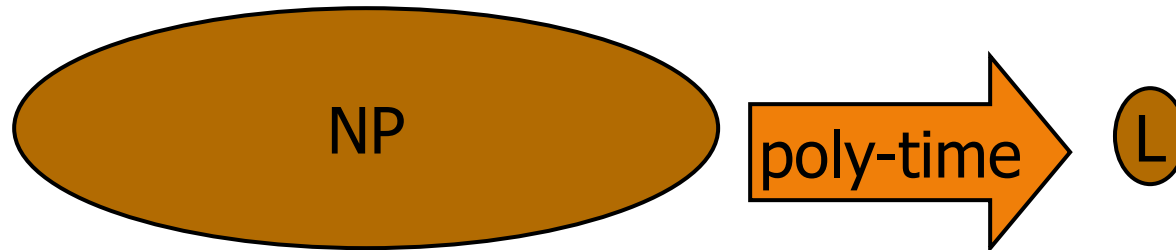
CIRCUIT-SAT is in NP

- ◆ Non-deterministically choose a set of inputs and the outcome of every gate, then test each gate's I/O.



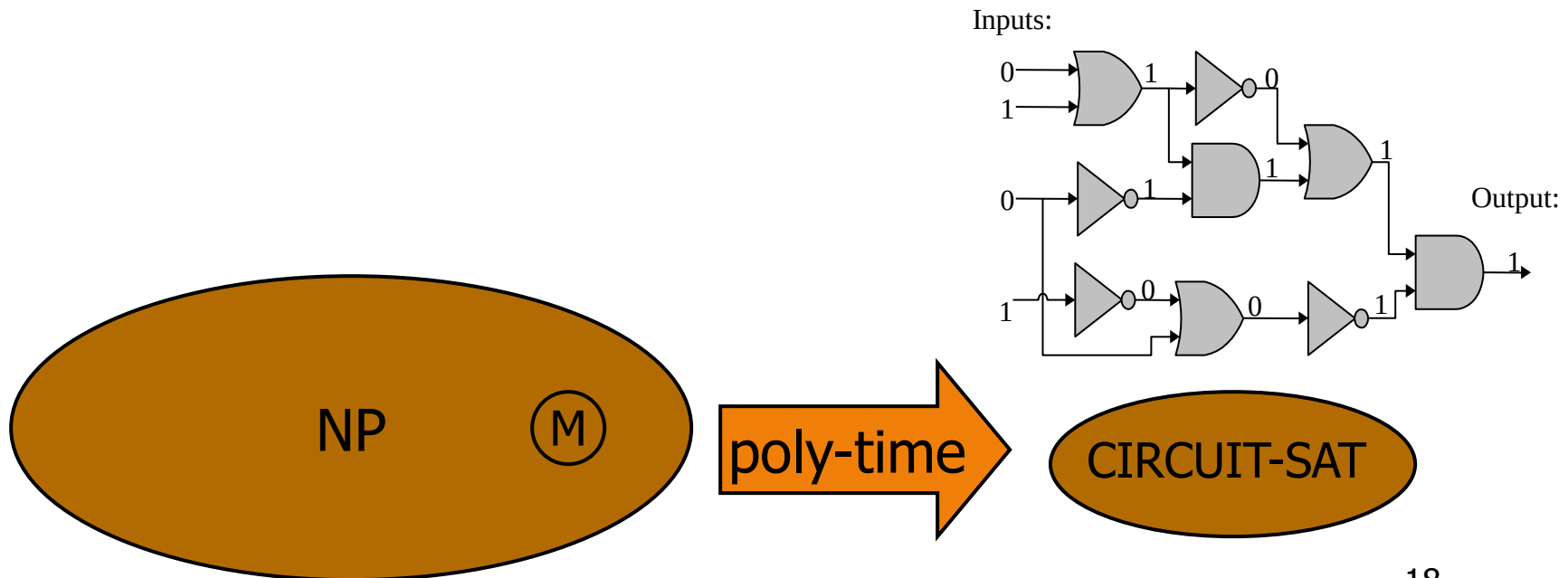
NP-Hard & NP-Complete

- ◆ A problem (language) L is **NP-hard** if every problem in NP can be reduced to L in polynomial time.
- ◆ That is, for each language M in NP, we can take an input x for M , **transform** it in polynomial time to an input x' for L such that x is in M if and only if x' is in L .
- ◆ L is **NP-complete** if it's in NP and is NP-hard.



Cook-Levin Theorem

- ◆ Proves CIRCUIT-SAT is NP-complete
- ◆ Shows that a polynomial-time reduction exists to SAT for any problem in NP



Transitivity of Reducibility



◆ If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$.

- An input x for A can be converted to x' for B , such that x is in A if and only if x' is in B . Likewise, for B to C .
- Convert x' into x'' for C such that x' is in B if x'' is in C .
- Hence, if x is in A , x' is in B , and x'' is in C .
- Likewise, if x'' is in C , x' is in B , and x is in A .
- Thus, $A \rightarrow C$, since polynomials are closed under composition.

◆ Types of reductions:

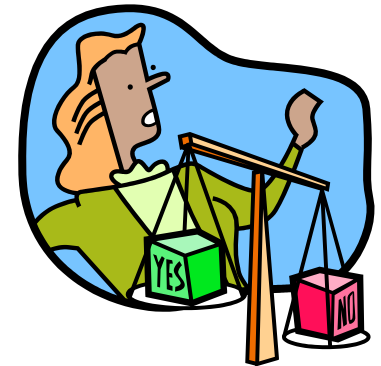
- **Local replacement:** Show $A \rightarrow B$ by dividing an input to A into components and show how each component can be converted to a component for B .
- **Component design:** Show $A \rightarrow B$ by building special components for an input of B that enforce properties needed for A , such as “choice” or “evaluate.”

SAT



- ◆ A Boolean formula is a formula where the variables and operations are Boolean (0/1):
 - $(a+b+\neg d+e)(\neg a+\neg c)(\neg b+c+d+e)(a+\neg c+\neg e)$
 - OR: +, AND: (times), NOT: $\neg$
- ◆ SAT: Given a Boolean formula S , is S satisfiable, that is, can we assign 0's and 1's to the variables so that S is 1 ("true")?
 - Easy to see that CNF-SAT is in NP:
 - ◆ Non-deterministically choose an assignment of 0's and 1's to the variables and then evaluate each clause. If they are all 1 ("true"), then the formula is satisfiable.

SAT is NP-complete

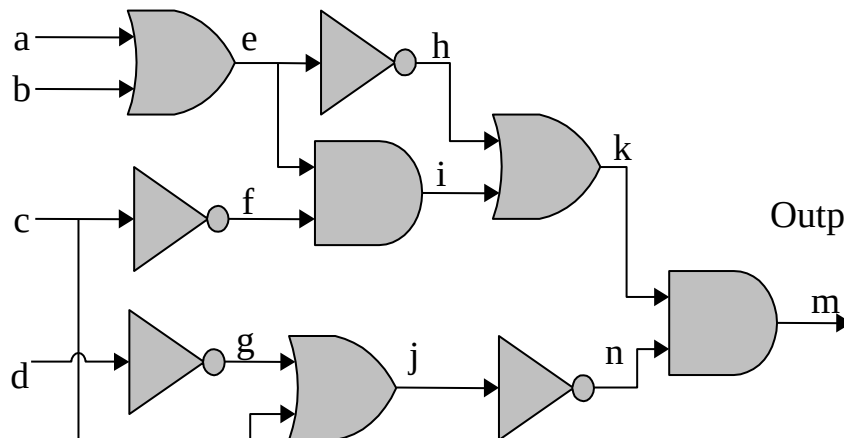


◆ Reduce CIRCUIT-SAT to SAT.

- Given a Boolean circuit, make a variable for every input and gate.
- Create a sub-formula for each gate, characterizing its effect. Form the formula as the output variable AND-ed with all these sub-formulas:

◆ Example:

Inputs: $m((a+b) \leftrightarrow e)(c \leftrightarrow \neg f)(d \leftrightarrow \neg g)(e \leftrightarrow \neg h)(ef \leftrightarrow i) \dots$



The formula is satisfiable
if and only if the
Boolean circuit
is satisfiable.



3SAT

- ◆ The 3SAT problem is NP-complete even if it is in CNF (conjunctive normal form) and every clause has just 3 literals (a variable or its negation):

- $(a+b+\neg d)(\neg a+\neg c+e)(\neg b+d+e)(a+\neg c+\neg e)$

- ◆ Reduction from SAT

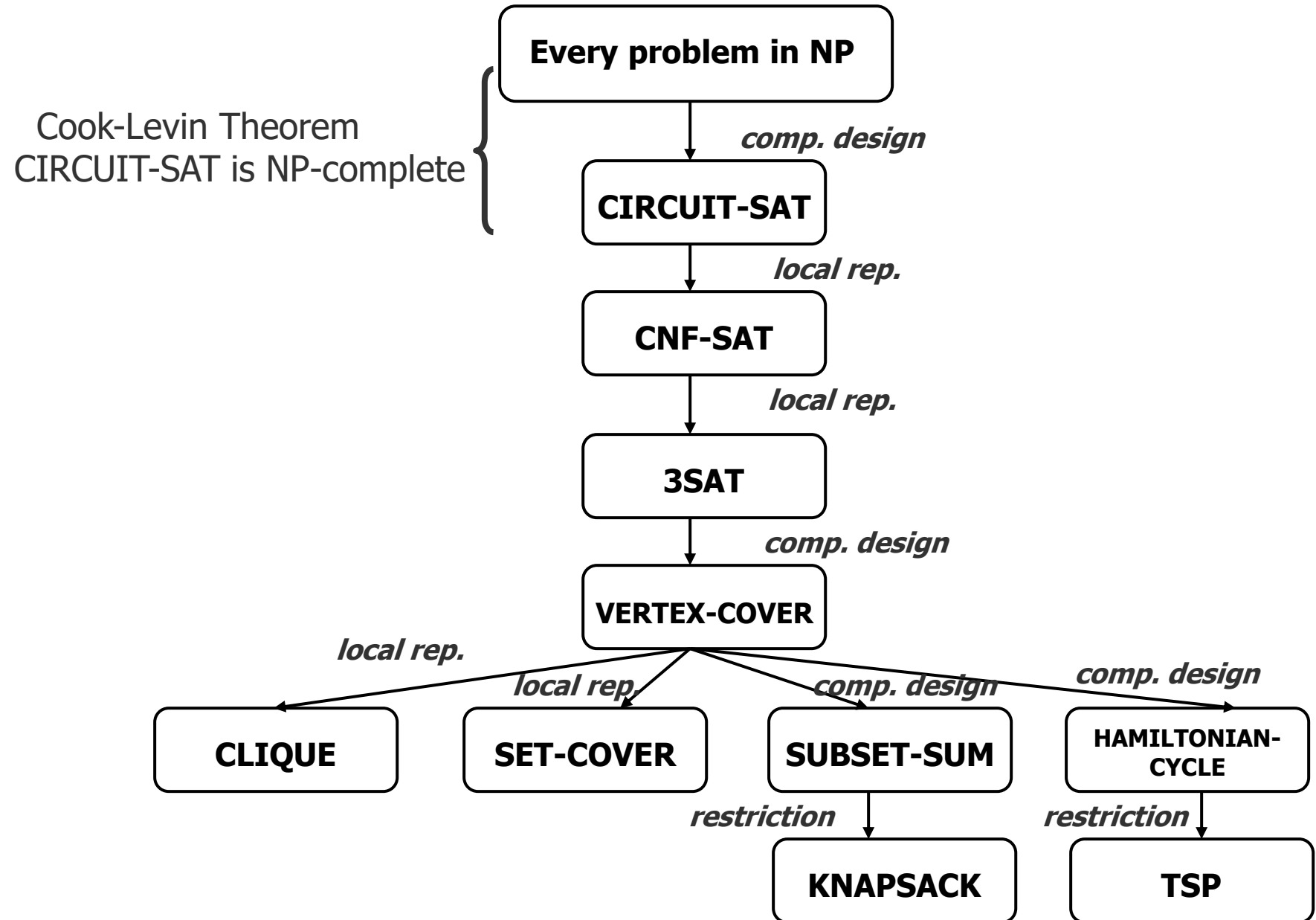
- ◆ DNF (disjunctive normal form):

$$B = a \cdot b \cdot c + \overset{-}{a} \cdot \overset{-}{b} \cdot c + \overset{-}{a} \cdot b \cdot \overset{-}{c} + a \cdot \overset{-}{b} \cdot \overset{-}{c}$$

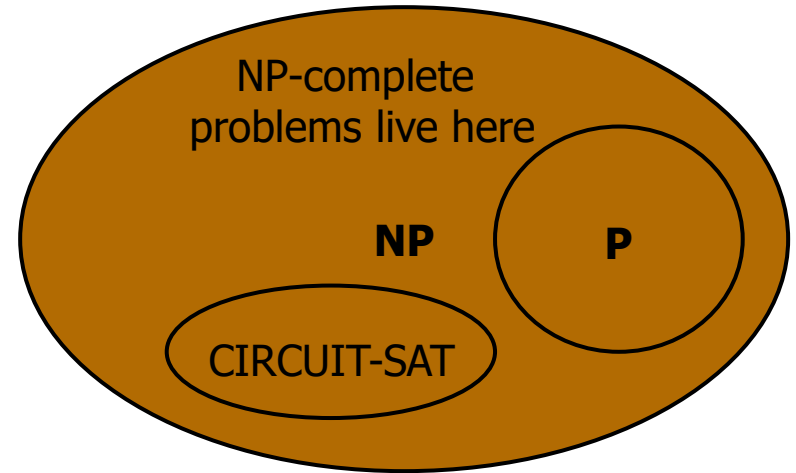
- ◆ CNF (conjunctive normal form):

$$B = (\overset{-}{a} + \overset{-}{b} + c) \cdot (\overset{-}{a} + b + \overset{-}{c}) \cdot (\overset{-}{a} + \overset{-}{b} + \overset{-}{c}) \cdot (a + b + \overset{-}{c})$$

Illustration of Reductions



Some Thoughts about P and NP



- ◆ Belief: P is a proper subset of NP .
- ◆ Implication: NP-complete problems are the hardest in NP .
- ◆ Why: Because if we could solve an NP-complete problem in polynomial time, we could solve every problem in NP in polynomial time.
- ◆ That is, if an NP-complete problem is solvable in polynomial time, then $P=NP$.
- ◆ Since so many people have attempted without success to find polynomial-time solutions to NP-complete problems, showing your problem is NP-complete is equivalent to showing that a lot of smart people have worked on your problem and found no polynomial-time algorithm.